

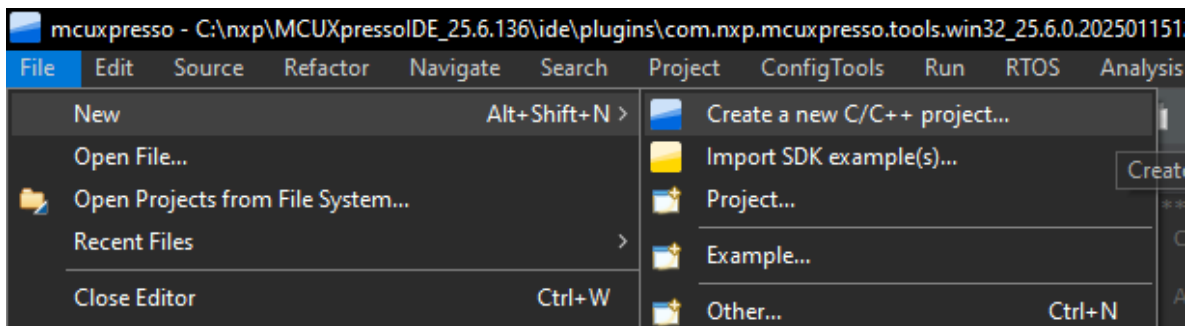
INSTRUCTIVO CONFIGURACIÓN DE PERIFÉRICOS Y MÓDULOS ÚTILES MCXN947

V1.0

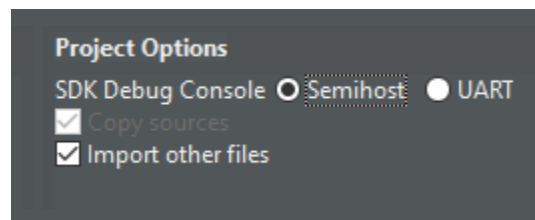
Índice

Creación del proyecto.....	2
CTIMER	5
GPIO	8
ADC	12
DAC	16

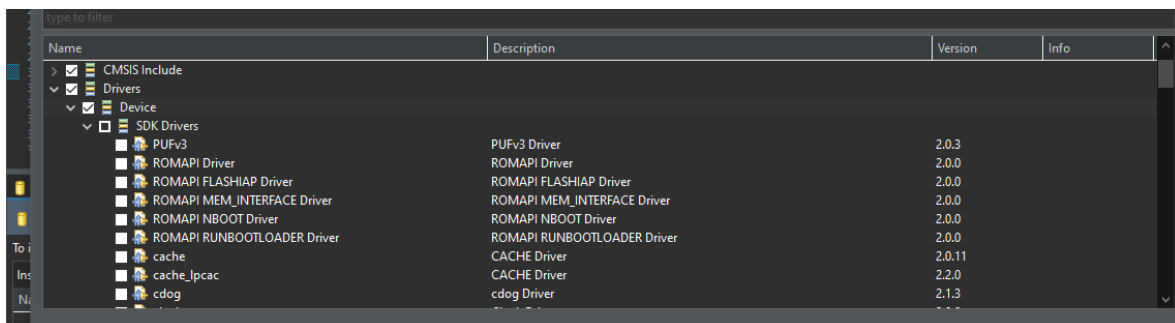
Creación del proyecto



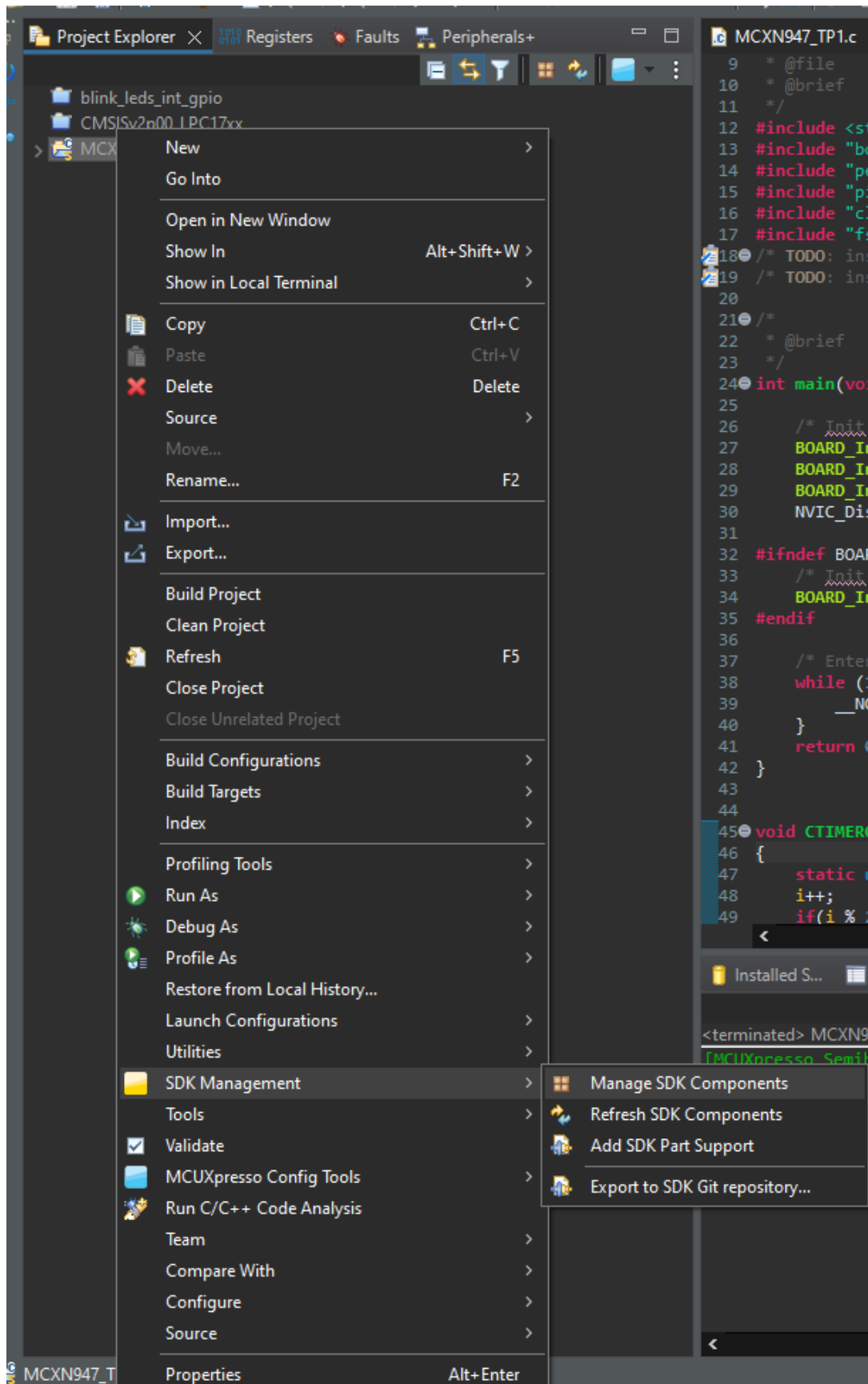
Para mayor simplicidad en el uso del printf() por consola, seleccionar **Semihost**:



En *SDK Drivers* seleccionar los periféricos a utilizar. En este instructivo se utilizará **LPADC**, **VREF**, **DAC**, **GPIO**, **CTIMER**. En *CMSIS Drivers* seleccionamos **CMSIS – DSP** para incluir la librería de DSP de ARM a utilizar en la materia.



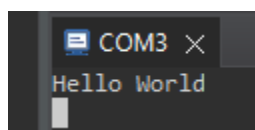
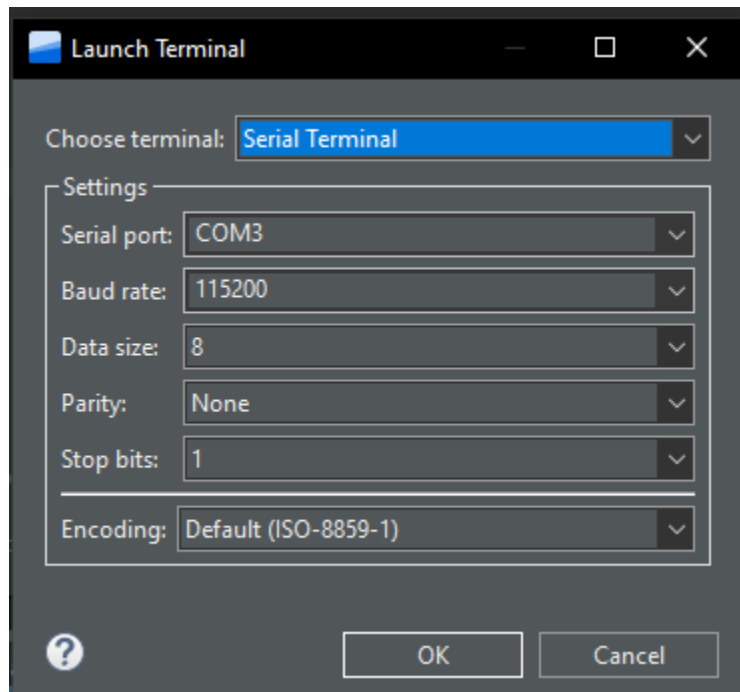
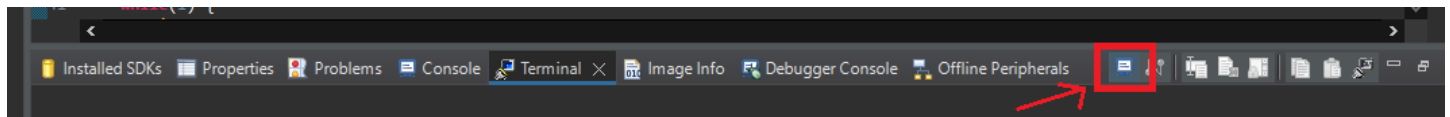
Si nos olvidamos de alguno, siempre podemos agregarlo haciendo click derecho en el proyecto, y seleccionando **SDK Managment**



Redirigimos el PRINTF del SKD a la función “printf” de C estándar, y también lo redirigimos junto con el “scanf” a la UART para depuración desde la PC.



Si hicimos todo bien, deberíamos poder ver el “Hello World” en una terminal luego de correr el código base que nos genera el IDE. Para verificar esto, abriremos una terminal:



Nota: Recordar que al cargar el código en la placa desde el modo debug, el program counter quedará pausado en la primera instrucción: **BOARD_InitBootPins();**

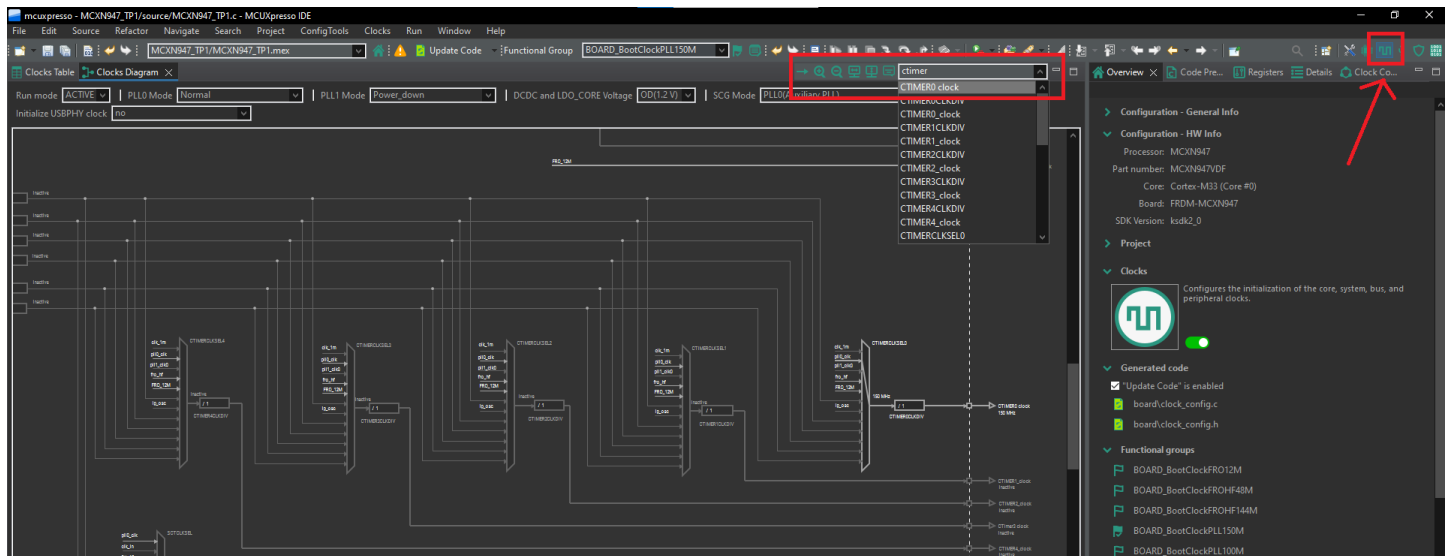


Deberemos presionar en “Resume” para que siga corriendo:

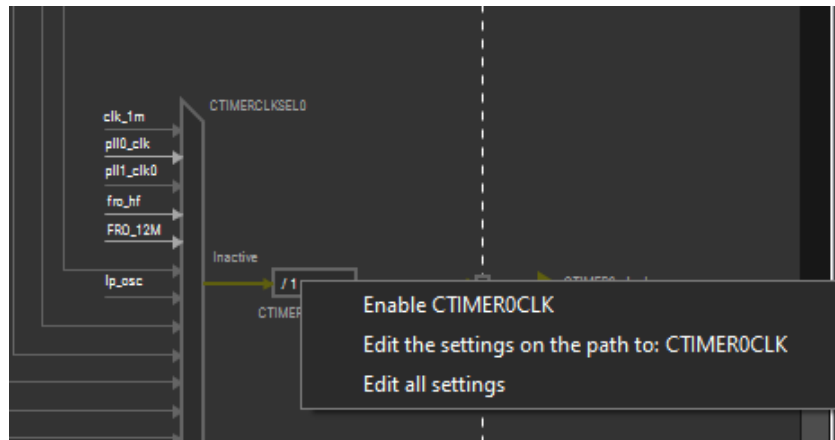


CTIMER

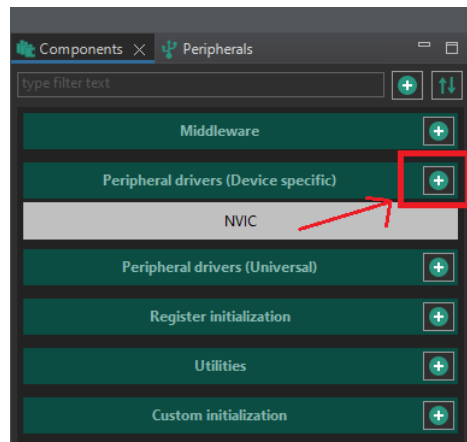
Primero debemos asignarle un clock al periférico **CTIMER0**, que es el que utilizaremos en este ejemplo. Nos dirigimos a la sección de clocks del configTools y buscamos **CTIMER0 clock**



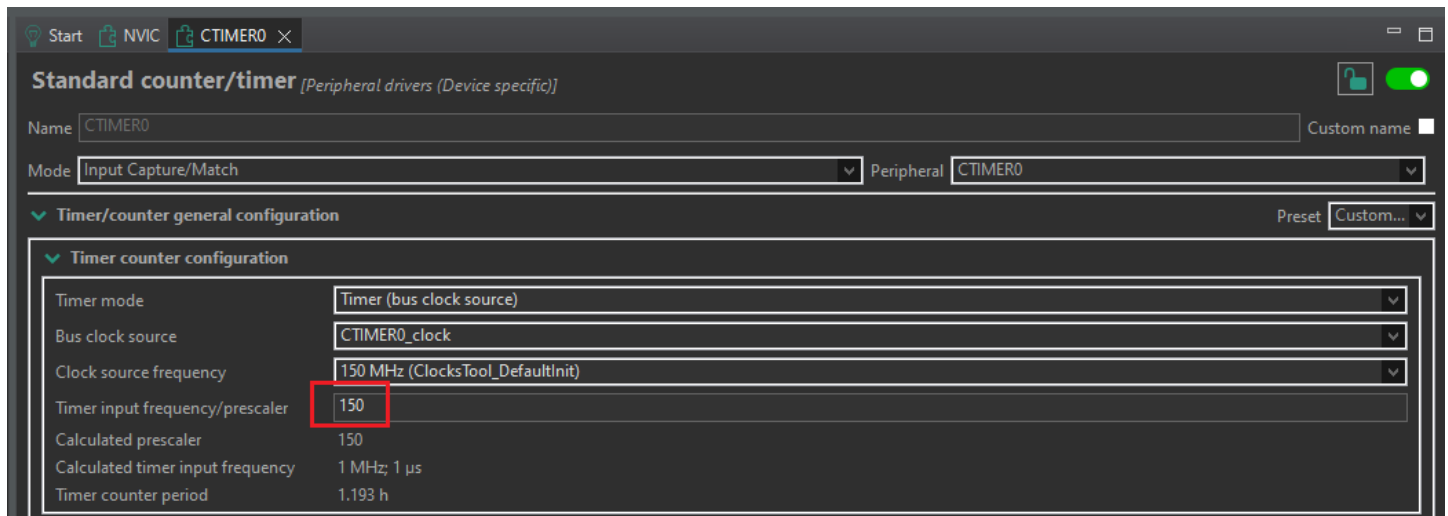
Seleccionamos una clock source (pll0_clk 150MHz en este ejemplo) y habilitamos el CTIMER0 clock



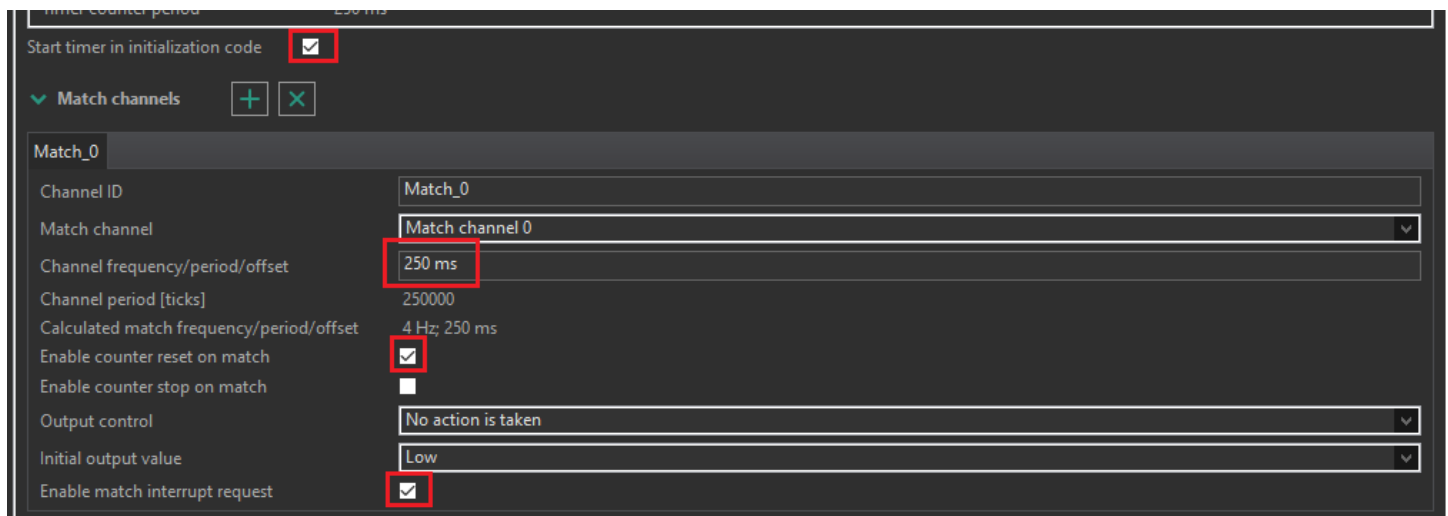
Ahora vamos a “Peripherals” y agregamos el CTIMER0



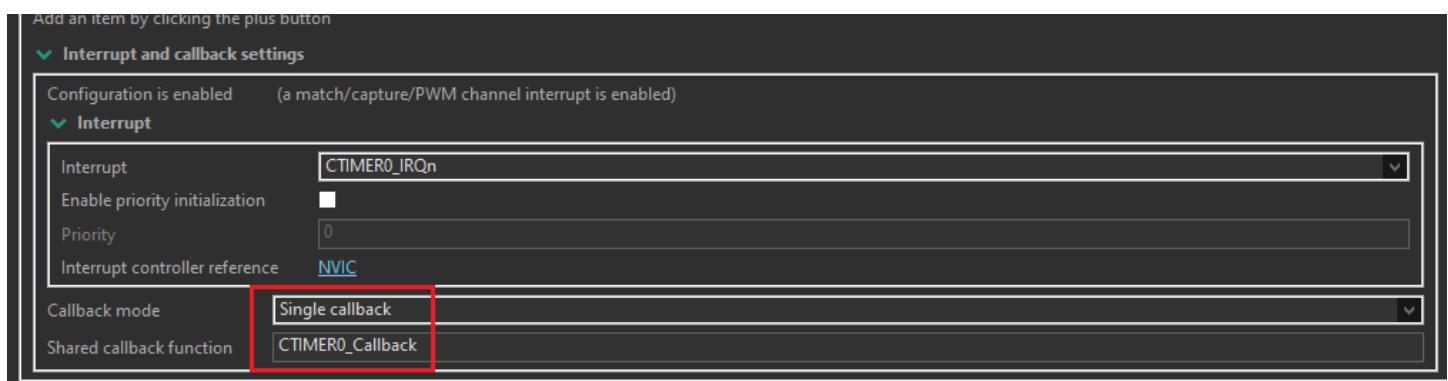
Colocamos 150 en “Timer input frequency/prescaler” para que el timer desborde cada 1 us



Luego habilitamos el timer en el código de inicialización y agregamos un **Match channel** con un **Channel frequency/period/offset** de 250ms. Tildamos en **Enable counter reset on match** para que se resetee al alcanzar el valor deseado, y habilitamos la petición de interrupciones.



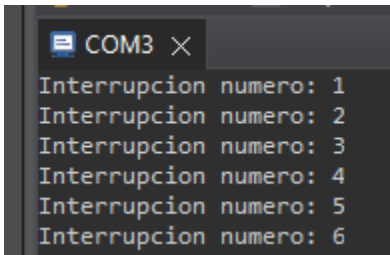
Utilizaremos solo un canal de match, por lo que en las configuraciones de interrupción pondremos modo **Single callback** y le pondremos un nombre a la función de callback que utilizaremos en el código



Ahora presionamos **update code** para actualizar estos cambios. Pegamos esta mínima función de interrupción:

```
void CTIMER0_Callback(uint32_t flags)
{
    static uint8_t i = 0;
    i++;
    PRINTF("Interrupcion numero: %u\r\n", i);
}
```

Y deberíamos ver en la terminal esto (cada 250ms):



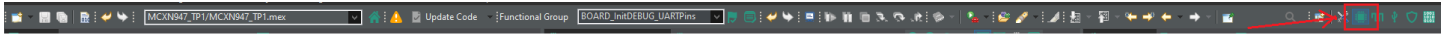
A screenshot of a terminal window titled "COM3" with a close button. The terminal displays six lines of text, each representing an interrupt event: "Interrupcion numero: 1" through "Interrupcion numero: 6". The text is in a monospaced font, and the lines are stacked vertically.

```
COM3 X
Interrupcion numero: 1
Interrupcion numero: 2
Interrupcion numero: 3
Interrupcion numero: 4
Interrupcion numero: 5
Interrupcion numero: 6
```

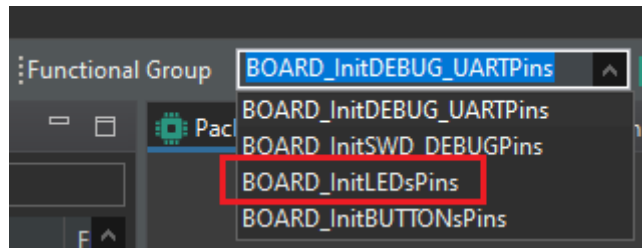
GPIO

Primero haremos blinkear el led de la placa cada 250ms, y luego mediante interrupción por GPIO (botón de la placa) cambiaremos el color del mismo.

Nos dirigimos a la sección **Pins** del configTools

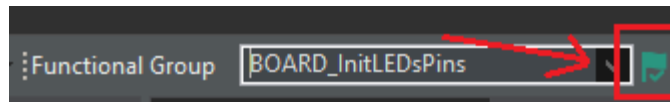


Seleccionamos el grupo funcional **BOARD_InitLEDsPins**



Es muy importante que la bandera verde al lado de nuestra selección del grupo funcional esté marcada, así inicializa la configuración seleccionada en el grupo funcional desde la función de inicialización

BOARD_InitBootPins

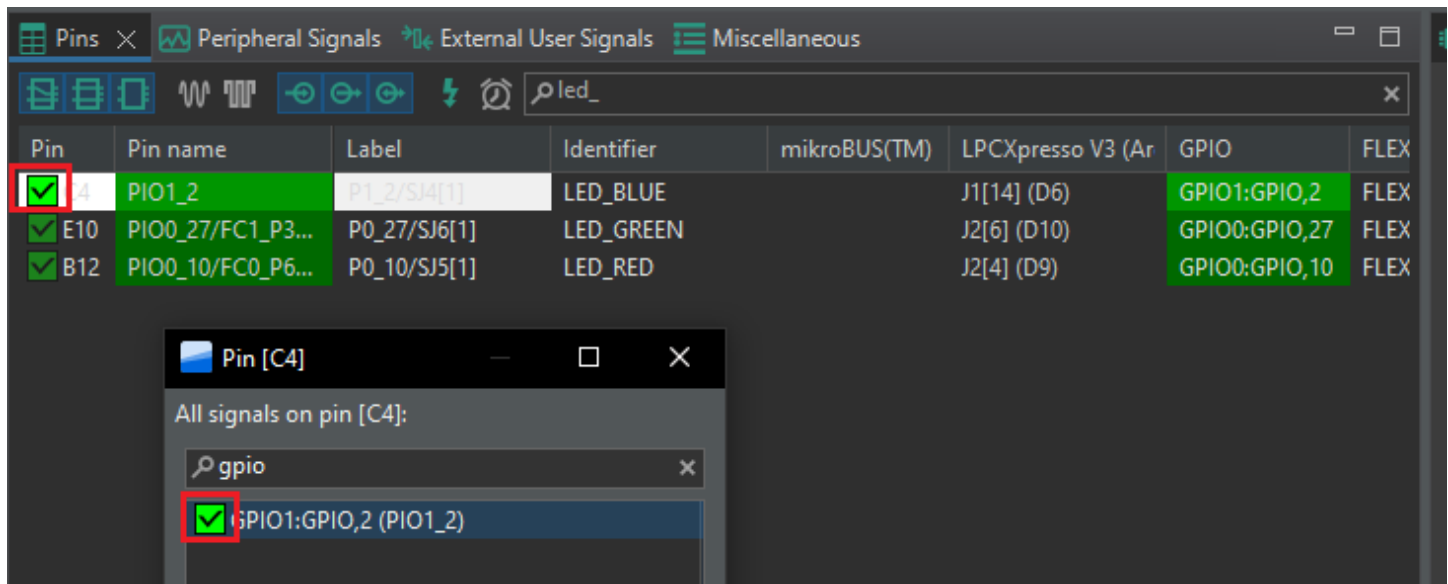


Buscamos en el buscador de pines “led_” para ver los pines conectados al led RGB de la placa

A screenshot of the 'Pins' table in configTools. The search bar at the top contains 'led_'. The table lists three pins: C4 (LED_BLUE), E10 (LED_GREEN), and B12 (LED_RED). Each row has a green checkmark in the first column. The 'GPIO' column shows the pin configuration for each LED.

Pin	Pin name	Label	Identifier	mikroBUS(TM)	LPCXpresso V3 (Ar	GPIO	FLEX
✓ C4	PIO1_2/TRIG_OU...	P1_2/SJ4[1]	LED_BLUE	J1[14] (D6)		GPIO1:GPIO,2	FLEX
✓ E10	PIO0_27/FC1_P3...	P0_27/SJ6[1]	LED_GREEN	J2[6] (D10)		GPIO0:GPIO,27	FLEX
✓ B12	PIO0_10/FC0_P6...	P0_10/SJ5[1]	LED_RED	J2[4] (D9)		GPIO0:GPIO,10	FLEX

Para habilitar por ejemplo el LED_BLUE, presionamos en el checkbox y en la ventana popup que nos aparece buscamos **GPIO**. Tildamos nuevamente el checkbox. Repetimos para los otros dos leds



Luego, el **Routing Details** debería quedarnos así:

Routing Details																
Routing Details for BOARD_Ini...																
#	Peripheral	Signal	Arrow	Routed pin/signal	Label	Identifier	Direction	GPIO initial state	GPIO interrupt selection	GPIO interrupt	EFT interrupt	Slew rate	Open drain	Drive strength	Pull select	Pull enable
C4	GPIO1	GPIO, 2	->	[C4] PIO1_2	P1_2/SJ4[1]	LED_BLUE	Output	Logical 0	n/a	n/a	Disabled	Fast	Disabled	Low	Pulldown	Disabled
B12	GPIO0	GPIO, 10	->	[B12] PIO0_10	P0_10/SJ5[1]	LED_RED	Output	Logical 0	n/a	n/a	Disabled	Fast	Disabled	Low	Pulldown	Disabled
E10	GPIO0	GPIO, 27	->	[E10] PIO0_27	P0_27/SJ6[1]	LED_GREEN	Output	Logical 0	n/a	n/a	Disabled	Fast	Disabled	Low	Pulldown	Disabled

Seleccionamos dirección de los leds **Output** y **GPIO initial state** como **Logical 1** ya que trabajan con lógica negativa. Dejamos las **Pulldown** por defecto.

Le damos “update code” y pegamos el siguiente código de prueba dentro del Callback del CTIMER0:

```

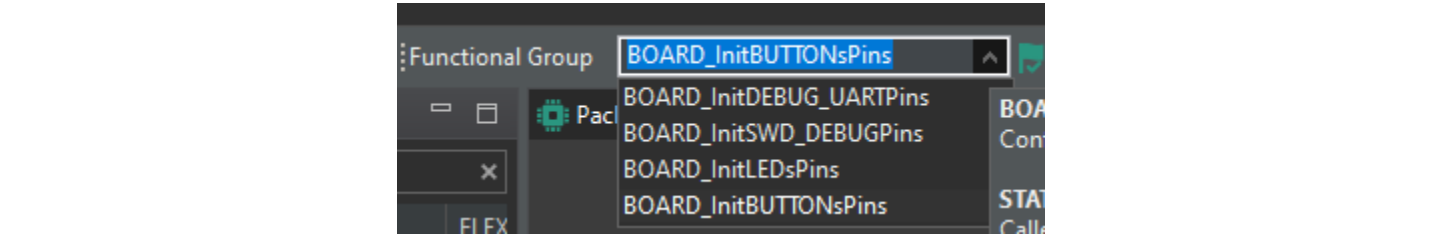
44 void CTIMER0_Callback(uint32_t flags)
45 {
46     static uint8_t i = 0;
47     i++;
48     if(i % 2){GPIO_PinWrite(BOARD_INITLEDSPINS_LED_GREEN_GPIO, BOARD_INITLEDSPINS_LED_GREEN_PIN, 0);}
49     else     {GPIO_PinWrite(BOARD_INITLEDSPINS_LED_GREEN_GPIO, BOARD_INITLEDSPINS_LED_GREEN_PIN, 1);}
50 }

```

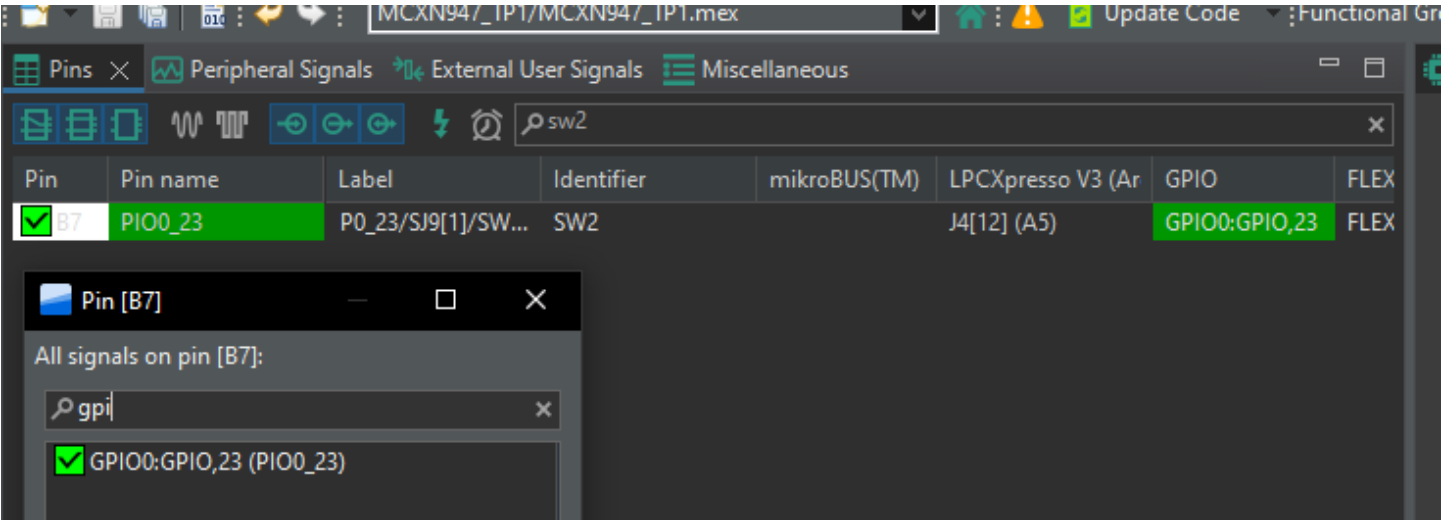
Deberíamos ver el led verde blinkear con un periodo de 500ms.

Nota: La función **GPIO_PinWrite(GPIO_Type *base, uint32_t pin, uint8_t output)** recibe un puntero base al periférico que debe escribir (GPIO0 para led rojo y verde y GPIO1 para led azul) y el número de pin que debemos escribir (27 para verde, 10 para rojo y 2 para azul). Estas definiciones se pueden encontrar en el header **pin_mux.h** que agregamos al proyecto mediante la inclusión del SDK para GPIO, y en un código escalable es conveniente redefinirlas en el main para mejor legibilidad.

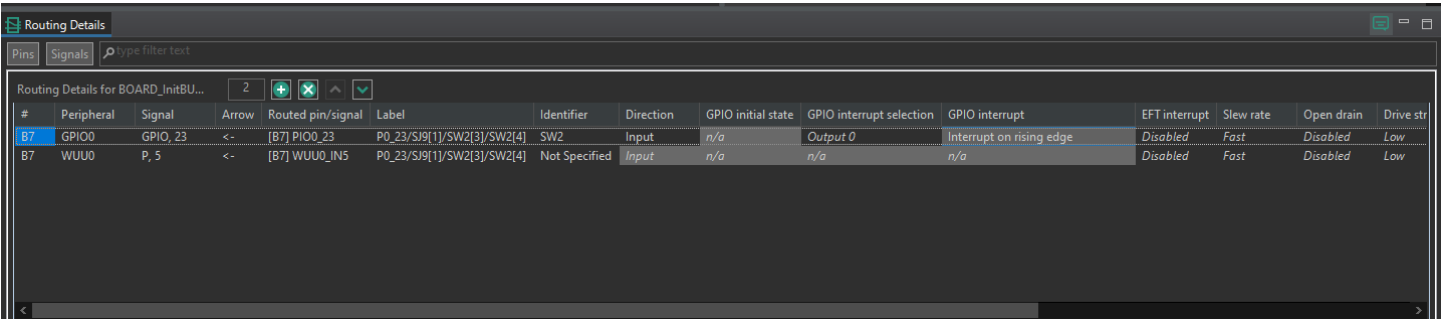
Para configurar uno de los botones como interrupción, seleccionamos el grupo funcional **BOARD_InitBUTTONsPins** (y marcamos la bandera)



Nuevamente, en **Pins** buscamos el botón que queremos utilizar. En este ejemplo usaremos el **SW2** y lo habilitamos como sigue:

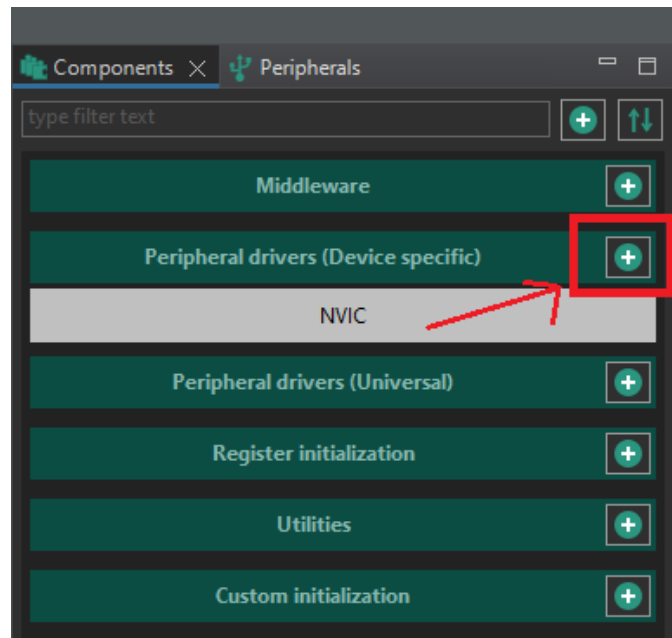


Luego, el **Routing Details** debería quedarnos así:

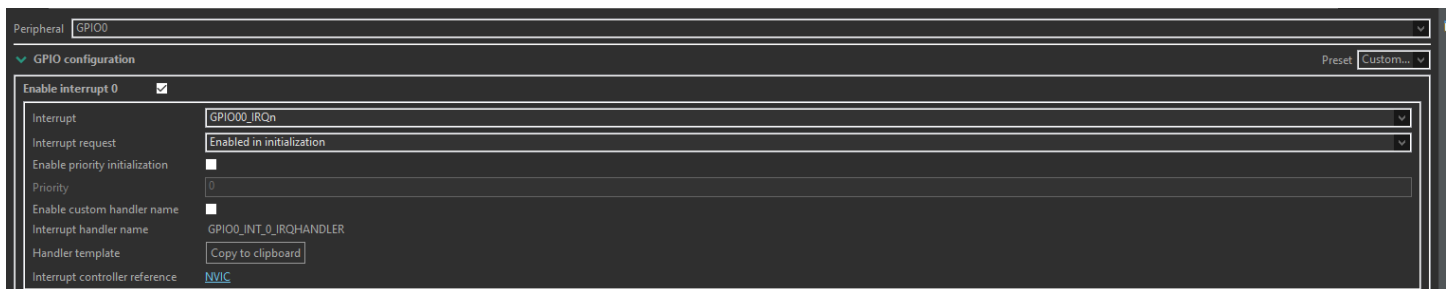


Debemos seleccionar **Interrupt on rising edge** en **GPIO interrupt**

Ahora, igual que con el CTIMER, nos dirigimos a **Components** de **Peripherals** y agregamos el **driver GPIO**



Habilitamos la interrupción 0 y listo. Copiamos al portapapeles el template del handler

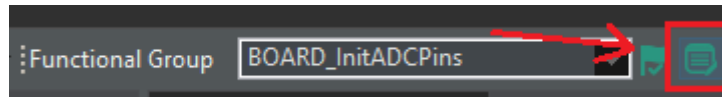


Pegamos esta pequeña rutina para cambiar el color del LED cada vez que se presiona un botón:

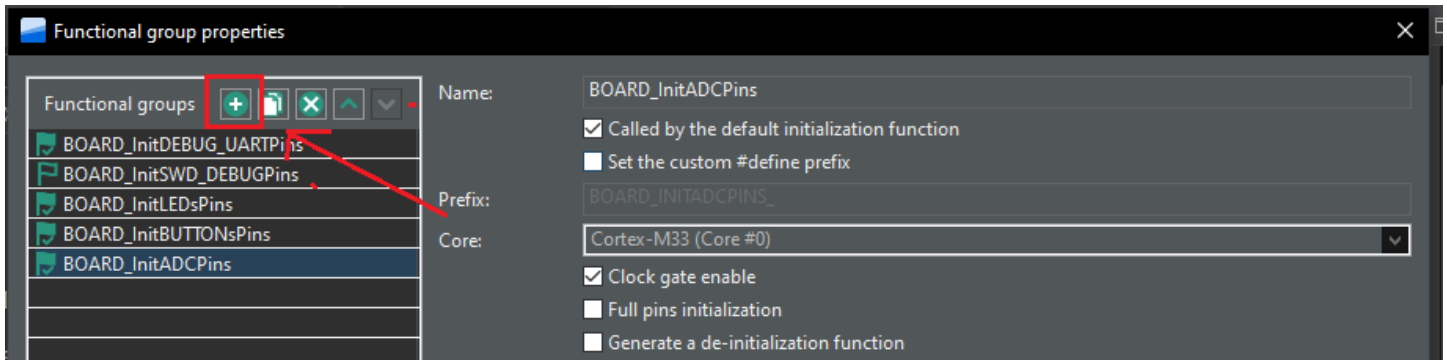
```
55 void GPIO0_INT_0_IRQHANDLER(void) {
56     uint32_t pin_flags0 = GPIO_GpioGetInterruptChannelFlags(GPIO0, 0U);
57
58     static uint8_t colour = 0;
59     colour++;
60
61     if (colour == 3){colour = 0;}
62     switch (colour) {
63     case 0:
64         GPIO_PinWrite(GPIO0, 27, 0);
65         GPIO_PinWrite(GPIO0, 10, 1);
66         GPIO_PinWrite(GPIO1, 2, 1);
67         break;
68     case 1:
69         GPIO_PinWrite(GPIO0, 27, 1);
70         GPIO_PinWrite(GPIO0, 10, 0);
71         GPIO_PinWrite(GPIO1, 2, 1);
72         break;
73     case 2:
74         GPIO_PinWrite(GPIO0, 27, 1);
75         GPIO_PinWrite(GPIO0, 10, 1);
76         GPIO_PinWrite(GPIO1, 2, 0);
77         break;
78     }
79
80     GPIO_GpioClearInterruptChannelFlags(GPIO0, pin_flags0, 0U);
81
82     #if defined __CORTEX_M && (__CORTEX_M == 4U)
83         __DSB();
84     #endif
85 }
86
```

ADC

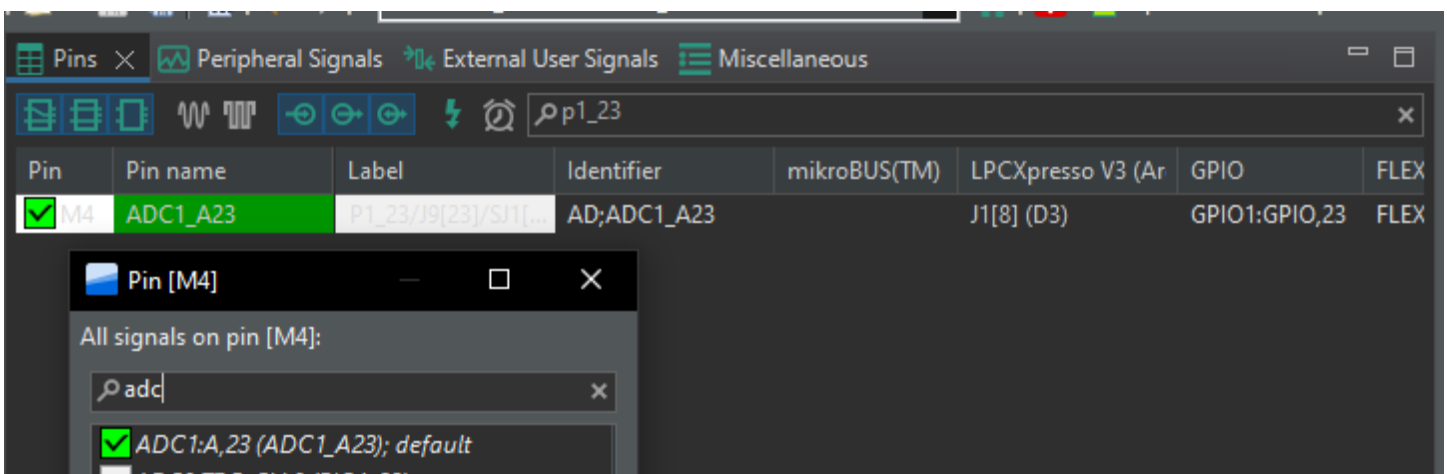
Vamos a **Pins** del configTools, y creamos un nuevo grupo funcional. Para esto, seleccionamos **Functional Group Properties**

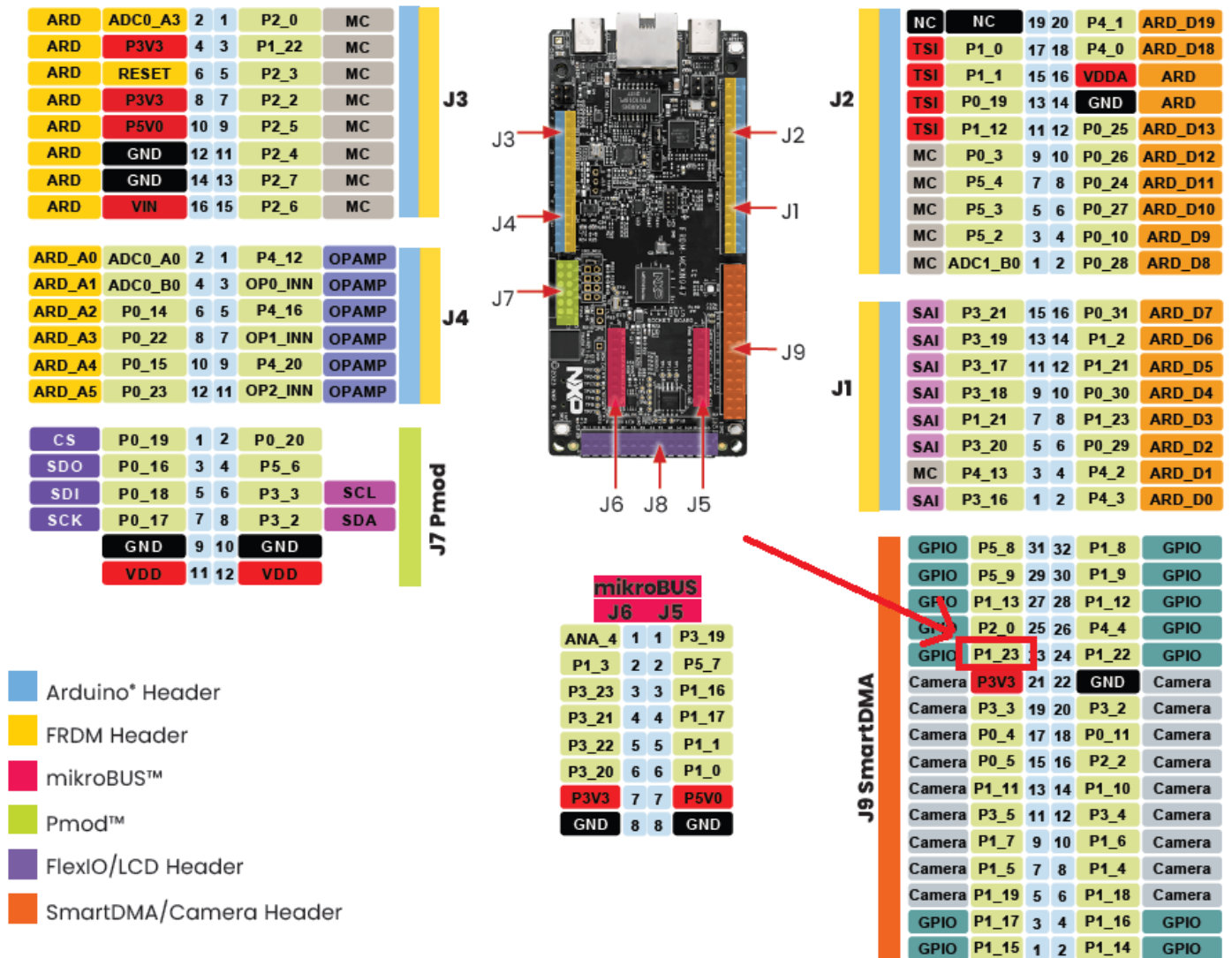


Seleccionamos **Add new** y lo llamamos **BOARD_InitADCPins** (para mantener la estructura propuesta por NxP)

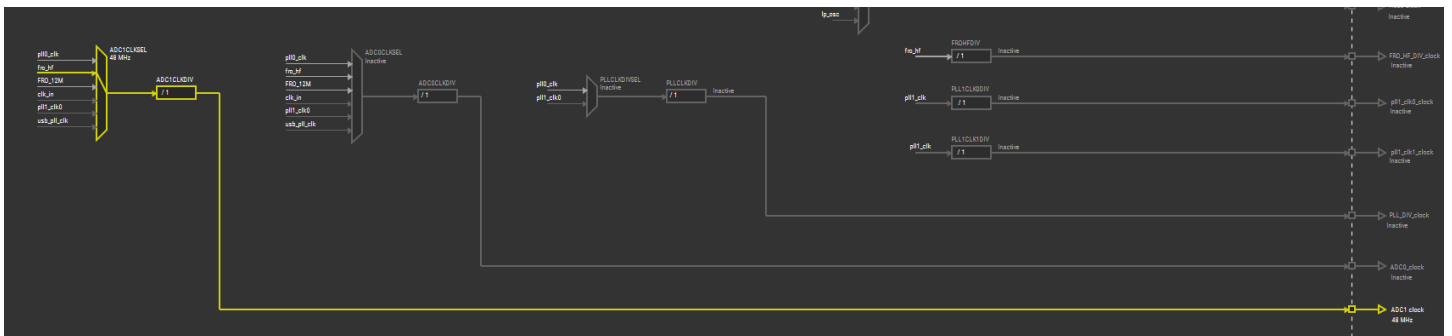


Seleccionamos el pin a utilizar como ADC. En este ejemplo, se utilizará el pin **P1_23** por su cercanía con GND y 3V3 ya que probaremos el ADC con un potenciómetro.

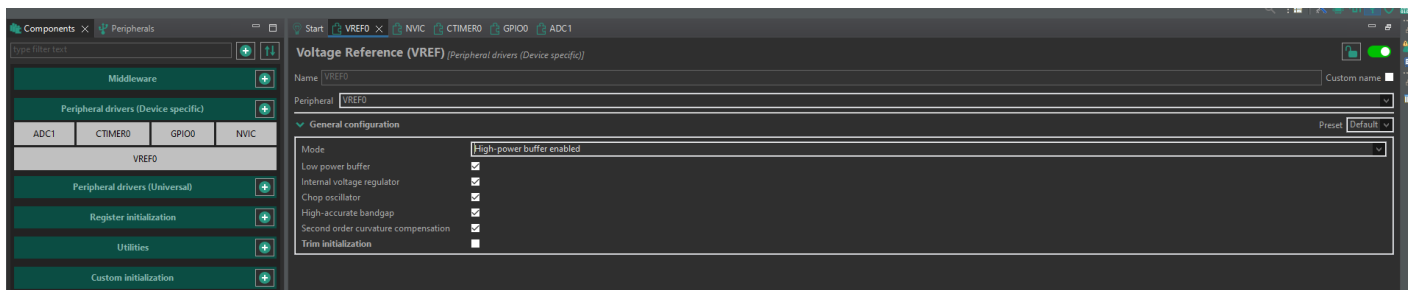




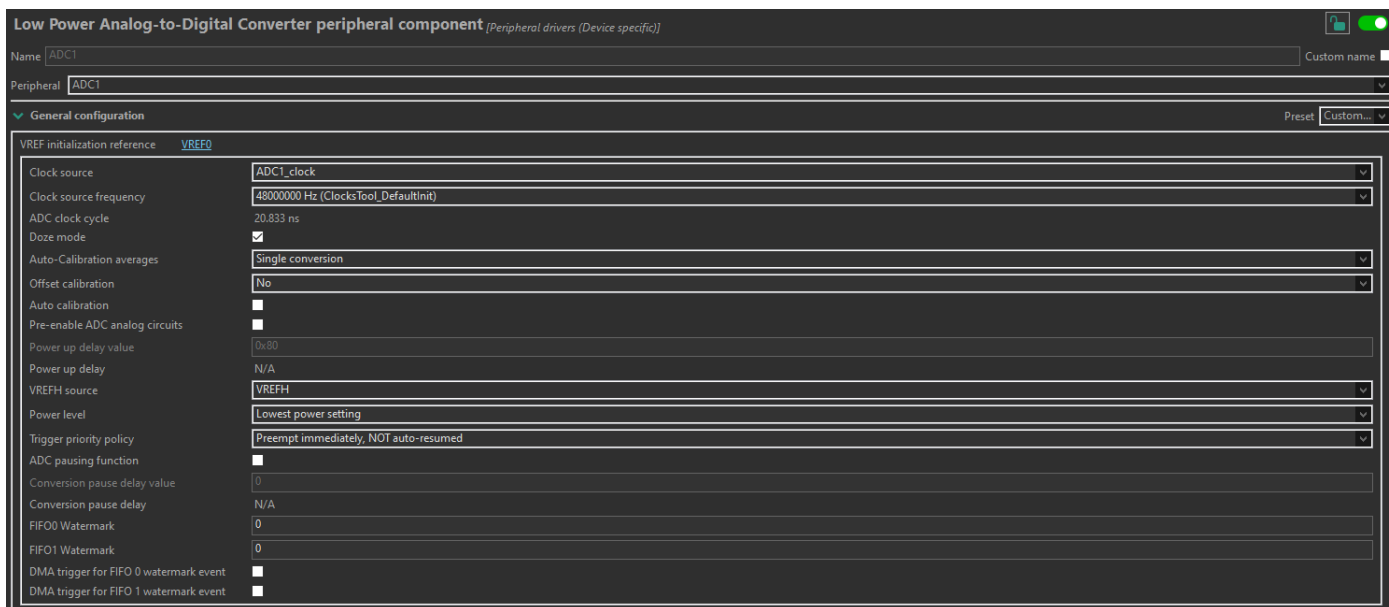
Nuevamente en la configuración del clocks del configtools, debemos seleccionar y activar un clock para el ADC que utilizaremos (en este ejemplo el ADC1)



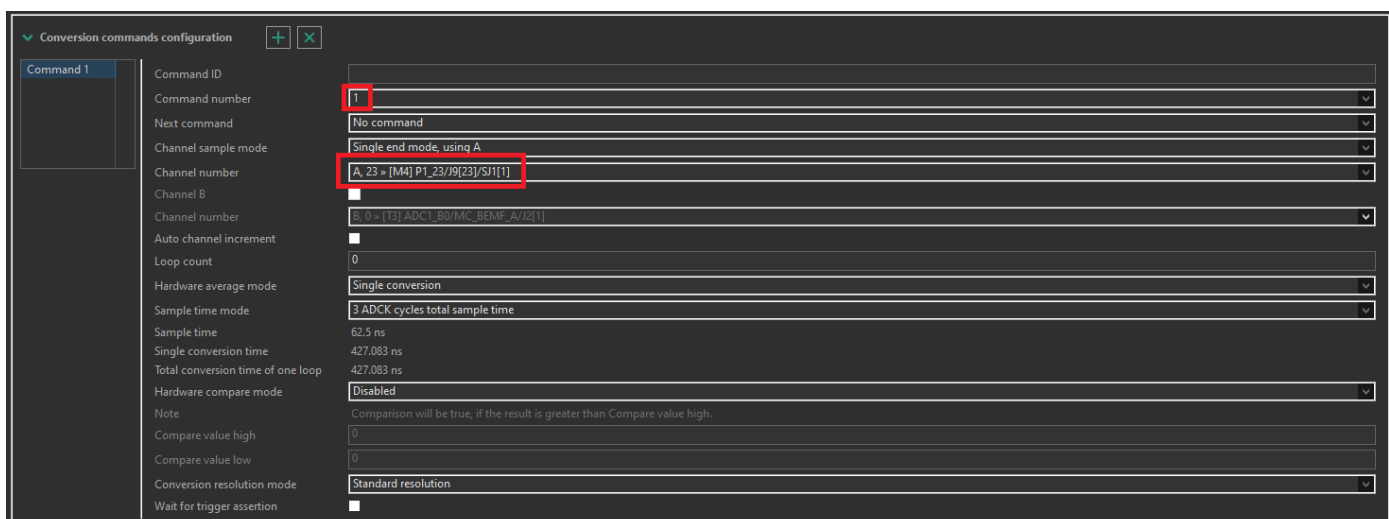
Ahora agregamos el periférico **VREF**



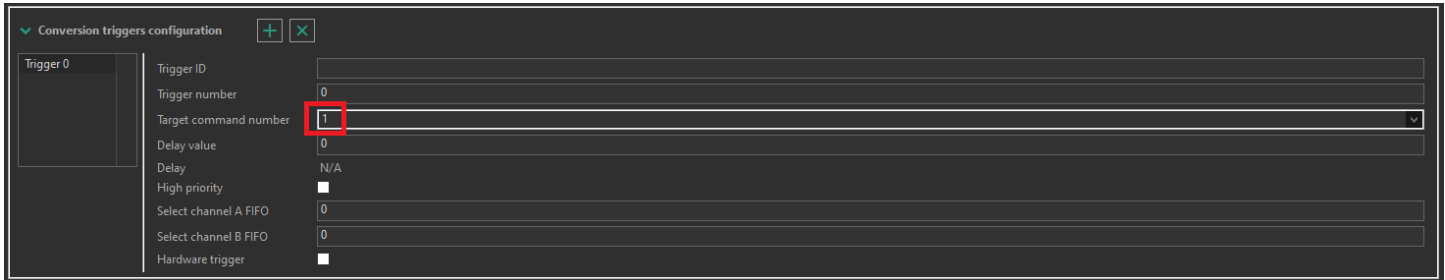
Ahora agregamos el periférico **ADC1**. Dejamos las selecciones por defecto de la siguiente parte:



En la sección de **Conversion commands configuration**, debemos agregar una configuracion para el canal que estamos utilizando. Al comando de tenemos que poner un número identificador, y debemos seleccionar el pin que utilizamos como ADC1 (P1_23 en este caso)



Luego, para disparar la conversión por software cada vez que entramos a una interrupción del CTIMER0, debemos agregar un **Conversion triggers configuration** y seleccionar el **Target command number** coincidente con el Command number que elegimos en el paso anterior.



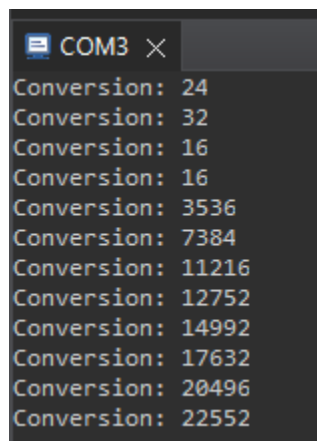
The screenshot shows a configuration window titled "Conversion triggers configuration". It has a sidebar with "Trigger 0" selected. The main area contains several fields: "Trigger ID" (empty), "Trigger number" (0), "Target command number" (1, highlighted with a red box), "Delay value" (0), "Delay" (N/A), "High priority" (checkbox), "Select channel A FIFO" (0), "Select channel B FIFO" (0), and "Hardware trigger" (checkbox).

Pegamos esta pequeña rutina (con el CTIMER0 debidamente configurado).

Agregar **#include "fsl_lpadc.h"**

```
void CTIMER0_Callback(uint32_t flags)
{
    static lpadc_conv_result_t res;
    LPADC_DoSoftwareTrigger(ADC1, 1U);
    LPADC_GetConvResult(ADC1, &res, 0U);
    PRINTF("Conversion: %u\r\n", res.convValue);
}
```

Una vez conectamos el potenciómetro, podremos ver en la terminal las conversiones que realiza el ADC1 con una resolución de 16 bits



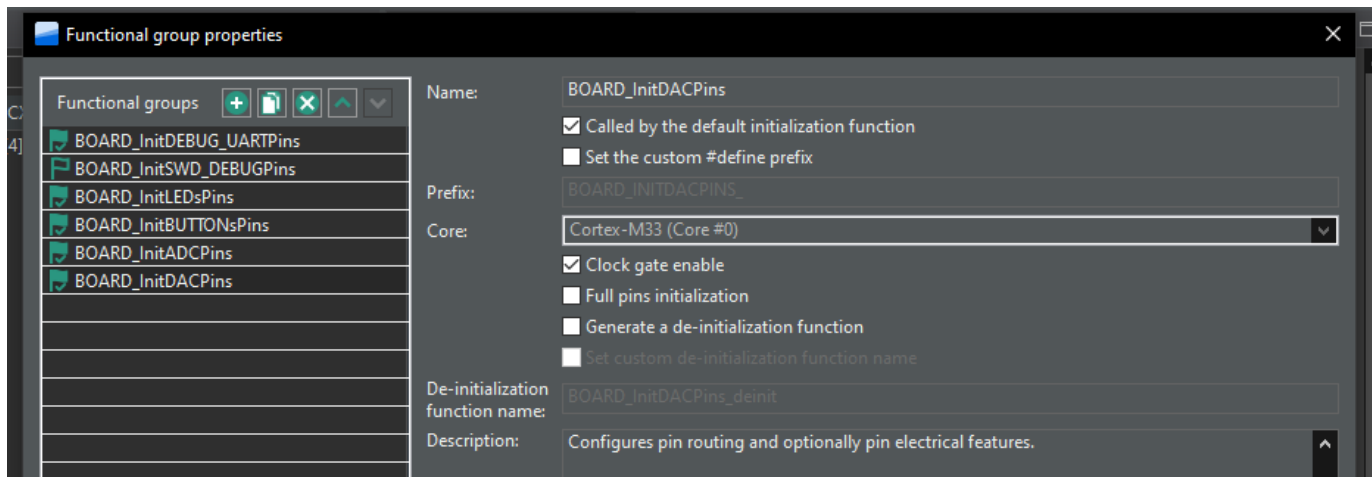
The terminal window shows the following output:

```
COM3 x
Conversion: 24
Conversion: 32
Conversion: 16
Conversion: 16
Conversion: 3536
Conversion: 7384
Conversion: 11216
Conversion: 12752
Conversion: 14992
Conversion: 17632
Conversion: 20496
Conversion: 22552
```

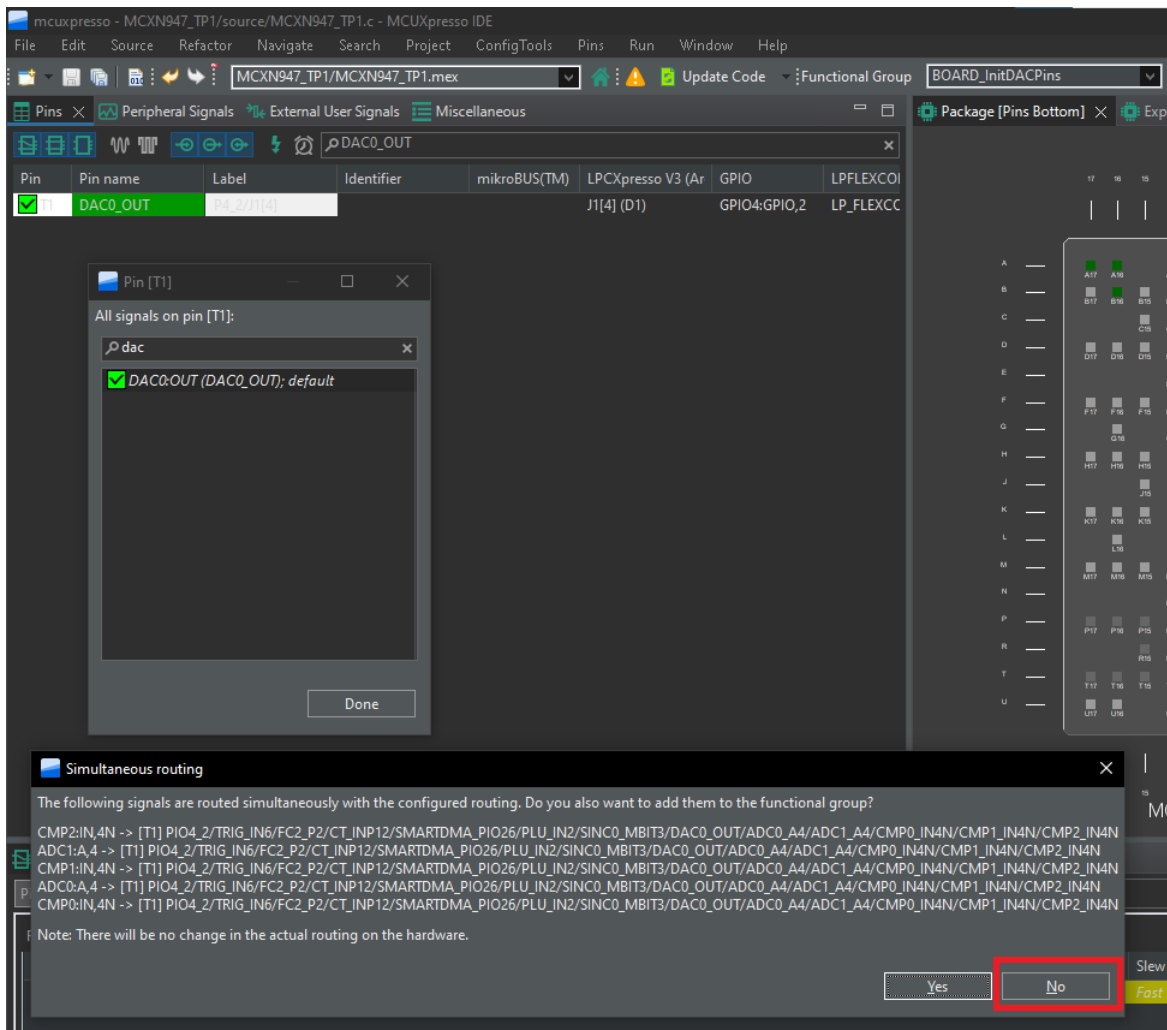
Nota: Esta es solo una forma simple de implementar el ADC, y no es la mas eficiente de todas. A altas frecuencias tendremos jitter al realizar el trigger por software. Una forma mas eficiente, sería realizar el trigger por hardware, o trabajar el ADC con interrupción y obtener el valor desde ahí cuando el CTIMER lo disponga, o trabajarlo con DMA.

DAC

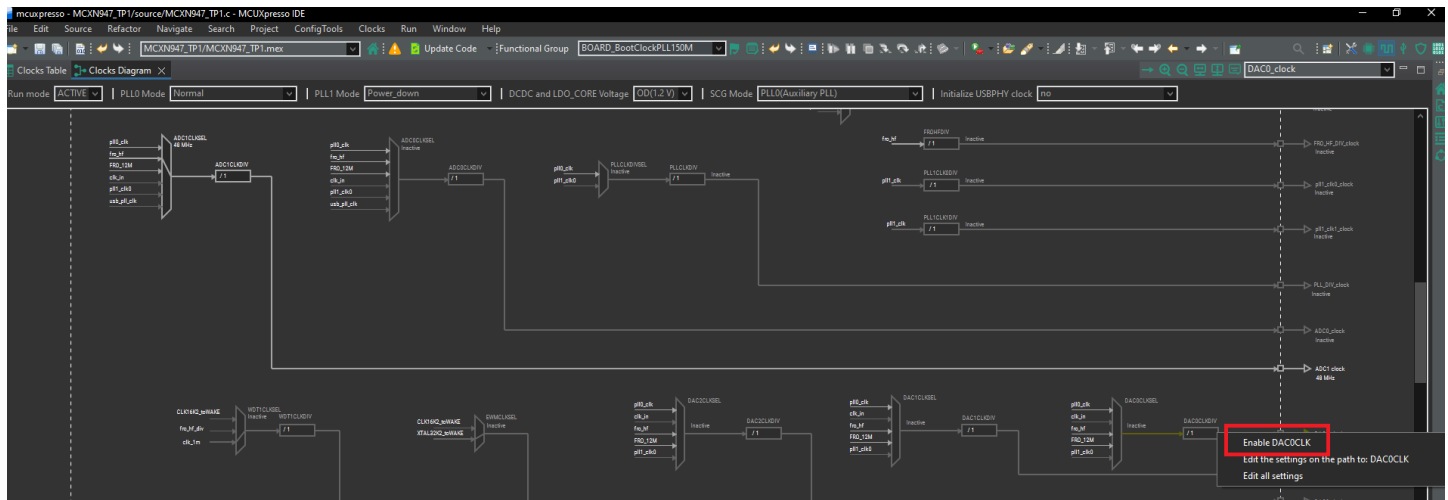
Primero nos dirigimos a **Pins** del configTools, y creamos un nuevo grupo funcional, igual que como hicimos con el ADC.



Luego, seleccionamos un pin que tenga la funcionalidad **DACX_OUT** y lo configuramos como sigue:



Luego le activamos un clock al DACX seleccionado:



Luego configuramos el periférico como sigue (revisar):

Con esta pequeña rutina, podremos sacar por el pin del DACX elegido (DAC0 en este caso) la señal analógica que estamos muestreando (revisar porque a velocidades menores a 1000us me tira hardfault). Debemos agregar **#include "fsl_dac.h"**.

```

56
57 void CTIMER0_Callback(uint32_t flags)
58 {
59     static lpadc_conv_result_t res;
60
61     if (LPADC_GetConvResult(ADC1, &res, 0U)) {
62         uint16_t adcValue = (uint16_t)res.convValue;
63         uint32_t dacValue = (adcValue >> 4) & 0xFFFF;
64
65         DAC_SetData(DAC0, dacValue);
66     }
67
68     LPADC_DoSoftwareTrigger(ADC1, 1U);
69 }
70

```

Deberíamos ver en el osciloscopio la señal muestreada con los pasos de cuantización:

